

A
Project Report
On

NegXus

Submitted in partial fulfilment of the requirement for the degree of
Bachelor of Technology

In
Computer Science and Engineering

By

Neeraj Suyal 2261388

Mayank Pandey 2261360

Sahil Kumar 2261504

Anin Hussain 2261093

Under the Guidance of

Mr. Prince Kumar

ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GRAPHIC ERA HILL UNIVERSITY, BHIMTAL CAMPUS

SATTAL ROAD, P.O. BHOWALI, DISTRICT-

NAINITAL-263132

2024-2025

STUDENT'S DECLARATION

We, **Neeraj Suyal, Mayank Pandey, Sahil Kumar** and **Anin Hussain** hereby declare the work, which is being presented in the project, entitled **NegXus** in partial fulfilment of the requirement for the award of the degree **Bachelor of Technology (B.Tech.)** in the session **2024-2025**, is an authentic record of my work carried out under the supervision of **Mr. Prince Kumar**. The matter embodied in this project has not been submitted by me for the award of any other degree.

Date: 30-05-25



Neeraj Suyal

Mayank Pandey

Sahil Kumar

Anin Hussain

CERTIFICATE

The project report entitled “NegXus” being submitted by **Neeraj Suyal, Mayank Pandey, Sahil Kumar, Anin Hussain of B.Tech.(CSE)** to **Graphic Era Hill University Bhimtal Campus** for the award of bonafide work carried out by them. They have worked under my guidance and supervision and fulfilled the requirement for the submission of a report.



Mr. Prince Kumar
(Project Guide)

Dr. Ankur Singh Bisht
(Head, CSE)

ACKNOWLEDGEMENT

We take immense pleasure in thanking the Honorable Director '**Prof. (Col.) Anil Nair (Retd.)**', GEHU Bhimtal Campus to permit me and carry out this project work with his excellent and optimistic supervision. This has all been possible due to his novel inspiration, able guidance, and useful suggestions that helped me to develop as a creative researcher and complete the research work, in time.

Words are inadequate in offering my thanks to GOD for providing me with everything that we need. We again want to extend thanks to our president '**Prof. (Dr.) Kamal Ghanshala**' for providing us with all infrastructure and facilities to work in need without which this work could not be possible.

Many thanks to '**Dr. Ankur Singh Bisht**' (Head, Department of Computer Science and Engineering, GEHU Bhimtal Campus), our project guide '**Mr. Prince Kumar**' (Assistant Professor, Department of Computer Science and Engineering, GEHU Bhimtal Campus) and other faculties for their insightful comments, constructive suggestions, valuable advice, and time in reviewing this report.

Finally, yet importantly, We would like to express my heartiest thanks to our beloved parents, for their moral support, affection, and blessings. We would also like to pay our sincere thanks to all my friends and well-wishers for their help and wishes for the successful completion of this project.

Neeraj Suyal, 2261388
Mayank Pandey, 2261360
Sahil Kumar, 2261504
Anin Hussain, 2261093

Abstract

The NegXus is a feature-rich, cross-platform chat application built using **Flutter**, **Firebase**, and **Cloudinary**, designed to deliver a seamless, real-time communication experience. With support for one-on-one chats, group chats, audio and video calls, file sharing, and user profile management, NegXus embodies the core principles of modern mobile app development—performance, scalability, and user-centric design.

The primary objective of NegXus is to gain hands-on experience with mobile-first, full-stack development by integrating Flutter’s powerful UI capabilities with Firebase’s real-time backend infrastructure. The project also leverages Cloudinary for optimized media storage and retrieval, ensuring efficient handling of images, audio, and other user-uploaded content.

NegXus supports a wide range of communication features, including:

- **Real-Time Chat and Group Chat:** Users can engage in instant one-on-one or group messaging, with real-time synchronization using Firebase Firestore.
- **Audio and Video Calling:** Seamless voice and video communication is facilitated through Firebase and WebRTC integrations.
- **File and Audio Sharing:** Users can send images, documents, and audio messages, with uploads handled securely via Cloudinary for optimized delivery and storage.
- **User Authentication and Profile Management:** Firebase Authentication ensures secure sign-up, login, and session handling. Users can update their profile pictures and personal information within the app.
- **Push Notifications:** Instant alerts for incoming messages and calls enhance user responsiveness and engagement.
- **Responsive and Modern UI:** Developed with Flutter’s widget-based architecture, NegXus features a responsive, smooth, and intuitive UI that runs consistently across Android and iOS platforms.
-

The architecture of NegXus emphasizes clean code practices, real-time synchronization, and efficient state management using providers or Bloc patterns (as applicable). Firebase Cloud Firestore serves as the primary NoSQL database, ensuring low-latency data updates, while Firebase Storage and Cloudinary handle larger media assets.

While the initial release focuses on core communication functionalities, the application is designed to scale and accommodate future enhancements, including status updates, message reactions, read receipts, end-to-end encryption, and integration with external services or AI-powered assistants.

NegXus stands as both a practical solution for real-time communication and a comprehensive learning tool that demonstrates the capabilities of Flutter and Firebase in building robust, real-world mobile applications. It showcases essential development practices such as modular UI design, secure authentication, real-time data handling, and optimized cloud-based media management.

TABLE OF CONTENTS

Declaration.....	i
Certificate.....	ii
Acknowledgement.....	iii
Abstract.....	iv
Table of Contents.....	v
CHAPTER 1 INTRODUCTION.....	6
1.1 Prologue.....	7
2.1 Background and Motivations.....	7
3.1 Problem Statement.....	8
4.1 Objectives and Research Methodology.....	8
5.1 Project Organization.....	9
CHAPTER 2 PHASES OF SOFTWARE DEVELOPMENT CYCLE	
1.1 Hardware Requirements.....	11
2.1 Software Requirements.....	11
CHAPTER 3 CODING OF FUNCTIONS.....	12
CHAPTER 4 SNAPSHOT.....	16
CHAPTER 5 LIMITATIONS (WITH PROJECT)	19
CHAPTER 6 ENHANCEMENTS.....	20
CHAPTER 7 CONCLUSION.....	21
REFERENCES.....	22

INTRODUCTION

1.1 Prologue :

In today's hyper-connected world, real-time communication platforms are fundamental to how individuals and communities interact. **NegXus: A Cross-Platform Chat Application** is a mobile-first solution built using **Flutter**, **Firebase**, and **Cloudinary**, designed to streamline instant messaging, media sharing, and voice/video communication.

NegXus addresses the growing demand for fast, secure, and intuitive communication tools by offering features such as one-on-one and group chats, audio/video calls, file transfers, and dynamic user management. This project serves not only as a functional communication platform but also as a comprehensive learning environment for developers to understand full-stack mobile app development using cloud services.

NegXus bridges theoretical concepts with real-world application, focusing on the integration of front-end UI/UX design, cloud database interaction, media optimization, and secure user authentication—all within a real-time, scalable architecture.

1.2 Background and Motivations :

The motivation behind **NegXus** is rooted in the desire to build a production-grade chat application while mastering key concepts of cross-platform app development and real-time communication systems. While applications like WhatsApp and Telegram dominate the messaging space, their technical complexities often remain inaccessible to learners.

With NegXus, we aim to:

- Gain in-depth experience with **Flutter** for mobile UI development.
- Master **Firebase services** such as Firestore, Authentication, Storage, and Cloud Functions.
- Implement **real-time messaging and calling** using Firebase and WebRTC.
- Manage **media uploads and optimization** via **Cloudinary**.
- Understand the principles of **secure authentication** and **role-based access**.
- Design responsive, consistent interfaces that work across Android and iOS.

NegXus serves as both a **real-time communication platform** and a **developer-centric learning tool**, laying the groundwork for advanced enhancements such as end-to-end encryption, message reactions, user status, AI chatbots, and performance analytics in future versions.

1.3 Problem Statement :

While real-time messaging platforms like **WhatsApp**, **Telegram**, and **Signal** are widely adopted, they often function as black boxes, with their internal architectures hidden from aspiring developers. This lack of transparency creates a disconnect between the theoretical concepts of mobile app development and the practical skills required to build secure, scalable, and feature-rich communication systems from the ground up.

The core problem addressed by this project is:

"How can we design and develop a scalable, real-time chat application using Flutter and Firebase that supports secure messaging, audio/video communication, media sharing, and user management, while reinforcing key principles of cross-platform mobile development and cloud integration?"

The solution must be **modular**, **robust**, and **extensible**, enabling the future integration of advanced features such as real-time notifications, AI-powered content recommendations, and third-party API integrations in the future.

1.4 Objectives and Research Methodology

Objectives

The primary objectives of the **NegXus** project are as follows:

- To design and develop a **feature-rich, real-time chat application** using **Flutter**, **Firebase**, and **Cloudinary**.
- To support:
 - **User Authentication and Profile Management:** Secure login system using Firebase Authentication, supporting email/password, Google sign-in, and user profile updates.
 - **One-on-One and Group Messaging:** Real-time chat support with seamless switching between personal and group conversations.
 - **Audio and Video Calls:** Integration of voice and video call functionality using WebRTC and Firebase services.
 - **Media and File Sharing:** Support for sharing images, documents, and audio files via Cloudinary for optimized media handling.
 - **Responsive and Cross-Platform UI:** A modern, user-friendly interface built with Flutter, compatible across Android and iOS.

- **Real-Time Updates:** Use of Firestore for instant message syncing, read receipts, and conversation updates.

Research Methodology

The project adopts a structured, modular, and agile research and development methodology:

- **Literature Review:** Analyzing existing messaging platforms (e.g., WhatsApp, Telegram, Discord) to understand their features, UX patterns, and communication flows.
- **Tool and Technology Selection:** Selecting **Flutter** for its cross-platform capabilities, **Firebase** for real-time services and backend infrastructure, and **Cloudinary** for optimized media management.
- **Component-Based Development:** Breaking the system into key modules—authentication, chat UI, real-time messaging, call functionality, and media sharing—and building each independently to ensure modularity and easier debugging.
- **Iterative Testing and Refinement:** Employing a test-driven approach for continuous testing, user feedback integration, and refining features through multiple iterations.
- **Version Control and Collaboration:** Utilizing Git for tracking progress, maintaining history, and ensuring smooth collaboration and deployment.

This methodology ensures a clear, maintainable, and scalable codebase while providing a strong foundation for future enhancements and learning.

1.5 Project Organization:

The **NegXus** project is organized into **seven core development phases**, each targeting essential features for building a modern, secure, and scalable real-time chat application. The architecture follows a modular structure, ensuring separation of concerns between user interface components, backend services, real-time communication logic, and media management. This enhances clarity, maintainability, and ease of future enhancements.

Phases of Development

1. User Authentication and Profile Management

- Implements secure user registration and login using **Firebase Authentication**.
- Supports multiple login methods (email/password, Google sign-in).
- Allows users to update profile information including display name and avatar.
- Manages sessions with token handling and logout functionality.

2. Real-Time Chat and Group Messaging

- Enables one-on-one and group conversations using **Cloud Firestore** for real-time syncing.
- Supports message status indicators (sent, delivered, read).
- Structures chat documents efficiently for scalable querying and updates.

3. Audio and Video Calling Module

- Integrates **WebRTC or compatible SDKs** for audio and video calling.
- Provides call initiation, reception, and termination capabilities.
- Ensures smooth call signaling using Firebase Cloud Functions and Firestore triggers.

4. Media, File, and Audio Sharing

- Supports sending and receiving images, videos, audio clips, and documents.
- Utilizes **Cloudinary** for optimized media storage and fast delivery.
- Enables in-chat previews and download access for shared files.

5. Responsive and Cross-Platform UI

- Designs a clean and interactive interface using **Flutter**, ensuring consistency across Android and iOS.
- Implements adaptive UI elements for dynamic chat views, call screens, and profile settings.
- Applies Material and Cupertino design patterns for native user experience.

6. Realtime Notifications and Sync

- Uses **Firebase Cloud Messaging (FCM)** for push notifications (e.g., new messages, missed calls).
- Implements background handlers and foreground listeners for real-time app updates.
- Plans for advanced features like in-app banners and smart replies.

7. Testing, Deployment, and Future Enhancements

- Conducts unit and widget testing for UI components and service layers.
- Leverages Firebase Hosting or Play Store/TestFlight for deployment.

- Prepares for future extensions such as:
 - End-to-End Encryption
 - Chatbots and AI-driven responses
 - Scheduled messages and message reactions
 - Third-party integrations (e.g., calendar, payments, Google Drive)

Each phase is developed incrementally to support scalability and reliability. The modular design of **NegXus** not only ensures seamless integration of upcoming features but also allows for rapid iteration and maintenance, making it a strong foundation for advanced communication platforms.



HARDWARE AND SOFTWARE REQUIREMENTS

2.1 Hardware Requirement :

The development and testing of the **NegXus** mobile chat application requires the following minimum hardware configuration to ensure smooth performance across emulators and real devices:

Component	Minimum Specification
Processor	Intel Core i3 6th Gen / AMD Ryzen 3 or higher
RAM	8 GB
Storage	10 GB of free disk space
Display	1366 x 768 resolution or higher
Input Devices	Keyboard and Mouse / Touchpad
Architecture	64-bit (x64) processor architecture
Mobile Device	(Optional) Android/iOS device for real testing

2.2 Software Requirements

The development stack and tools used for **NegXus** include the following:

Core Technologies & Tools:

- **Operating System:** Windows 10 / 11, macOS Monterey+, Ubuntu 20.04+
- **Mobile SDK:** Flutter SDK ($\geq 3.0.0$)
- **Programming Language:** Dart
- **IDE / Code Editor:** Visual Studio Code / Android Studio

Backend & Services:

- **Authentication & Real-Time Database:** Firebase (Authentication, Firestore)
- **Push Notifications:** Firebase Cloud Messaging (FCM)
- **Storage (Media Files):** Cloudinary
- **Hosting:**
 - **Frontend & App Files:** Firebase Hosting
 - **APK Builds & Releases:** GitHub (Release section or GitHub Actions Artifacts)

Development Tools:

- **Emulators:** Android Emulator / iOS Simulator
- **Version Control:** Git with GitHub
- **Package Manager:** pub.dev (for Flutter dependencies)
- **API & Debugging Tools:** Firebase Emulator Suite, Flutter DevTools
- **Browser Compatibility:** Google Chrome, Mozilla Firefox, Safari (for Firebase Console access and testing)

CODE:

```
final FlutterLocalNotificationsPlugin flutterLocalNotificationsPlugin = FlutterLocalNotificationsPlugin();

Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  await Firebase.initializeApp();
  print('Handling background message: ${message.messageId}');
}

final navigatorKey = GlobalKey<NavigatorState>();

// Create a Cloudinary instance and set your cloud name.

void main() async {
  WelcomePageString.cloudinary.config.urlConfig.secure = true;
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);

  const AndroidInitializationSettings androidInitSettings = AndroidInitializationSettings('@mipmap/ic_launcher');
  final InitializationSettings initSettings = InitializationSettings(android: androidInitSettings);
  await flutterLocalNotificationsPlugin.initialize(initSettings);
  ZegoUIKitPrebuiltCallInvitationService().setNavigatorKey(navigatorKey);

  await ZegoUIKit().initLog();

  runApp(const MyApp());
}

class AppController extends GetxController {
  RxString oldVersion = ''.obs;
  RxString currentVersion = ''.obs;
  RxString newAppUrl = ''.obs;

  void onInit() async {
    super.onInit();
    PackageInfo packageInfo = await PackageInfo.fromPlatform();
    currentVersion.value = packageInfo.version;
    print(currentVersion.value);
    checkLatestVersion();
  }

  Future<void> checkLatestVersion() async {
    const repositoryOwner = 'suyalneeraj19';
    const repositoryName = 'negxus_webb';
    final response = await http.get(Uri.parse(
      'https://api.github.com/repos/$repositoryOwner/$repositoryName/releases/latest',
    ));
    if (response.statusCode == 200) {
      final Map<String, dynamic> data = json.decode(response.body);
      final tagName = data['tag_name'];
      oldVersion.value = tagName;
      final assets = data['assets'] as List<dynamic>;
      for (final asset in assets) {
        final assetName = asset['name'];
        final assetDownloadUrl = asset['browser_download_url'];
        newAppUrl.value = assetDownloadUrl;
      }
    }
  }
}
```

```

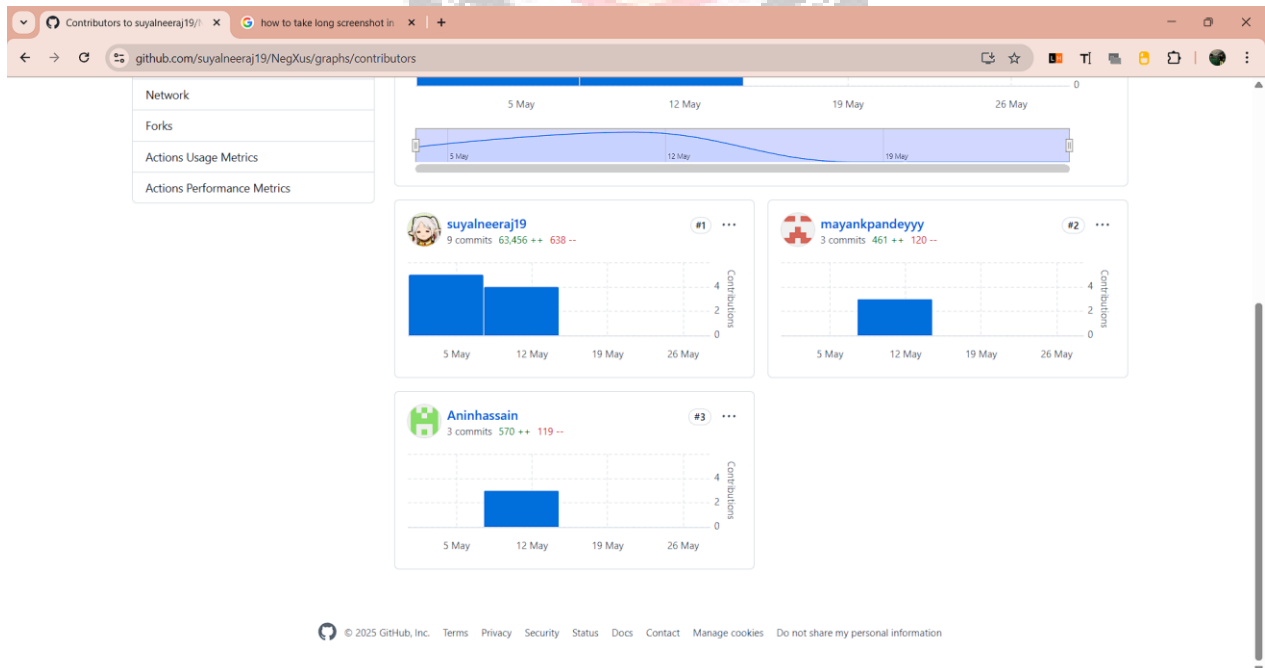
class VoiceMessagePlayer extends StatefulWidget {
  final String audioUrl;
  final bool isComming;
  const VoiceMessagePlayer({super.key, required this.audioUrl, required this.isComming});

  @override
  State<VoiceMessagePlayer> createState() => _VoiceMessagePlayerState();
}

class _VoiceMessagePlayerState extends State<VoiceMessagePlayer> {
  late AudioPlayer _player;
  late PlayerController _waveController;
  bool _isPlaying = false;
  Duration _duration = Duration.zero;
  Duration _position = Duration.zero;
  String? _localPath;
  bool _isLoading = true;
  bool _isError = false;

  @override
  void initState() {
    super.initState();
    _player = AudioPlayer();
    _waveController = PlayerController();
  }
}

```



```

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> with TickerProviderStateMixin {
  @override
  Widget build(BuildContext context) {
    TabController tabController = TabController(length: 3, vsync: this);
    ProfileController profileController = Get.put(ProfileController());
    ContactController contactController = Get.put(ContactController());
    ImagePickerController imagePickerController = Get.put(ImagePickerController());
    StatusController statusController = Get.put(StatusController());
    CallController callController = Get.put(CallController());
    AppController appController = Get.put(AppController());

    return Scaffold(
      appBar: AppBar(
        backgroundColor: Theme.of(context).colorScheme.primaryContainer,
        title: Text(
          AppString.appName,
          style: Theme.of(context).textTheme.headlineSmall,
        ), // Text
        leading: Padding(
          padding: const EdgeInsets.all(8.0),
          child: SvgPicture.asset(
            AssetsImage.appIconSVG,
          ), // SvgPicture.asset

```



```

class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

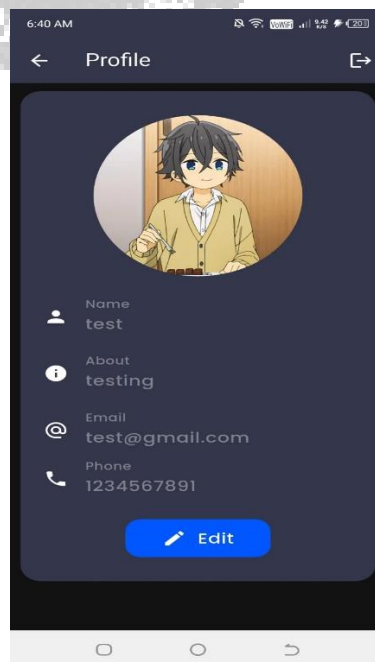
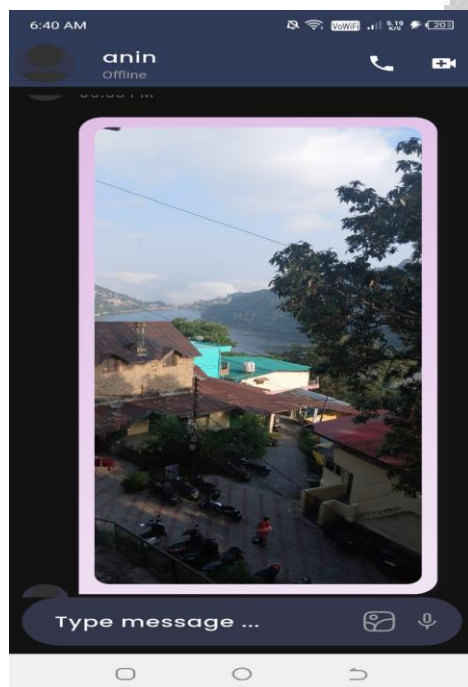
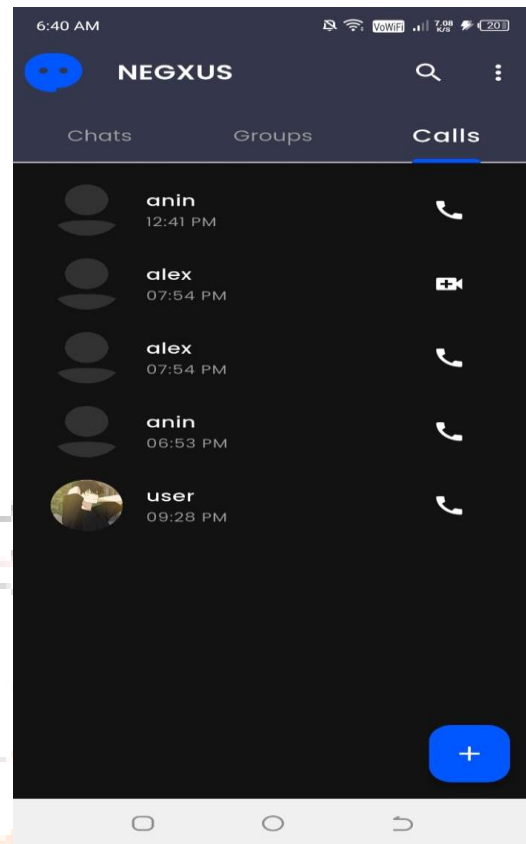
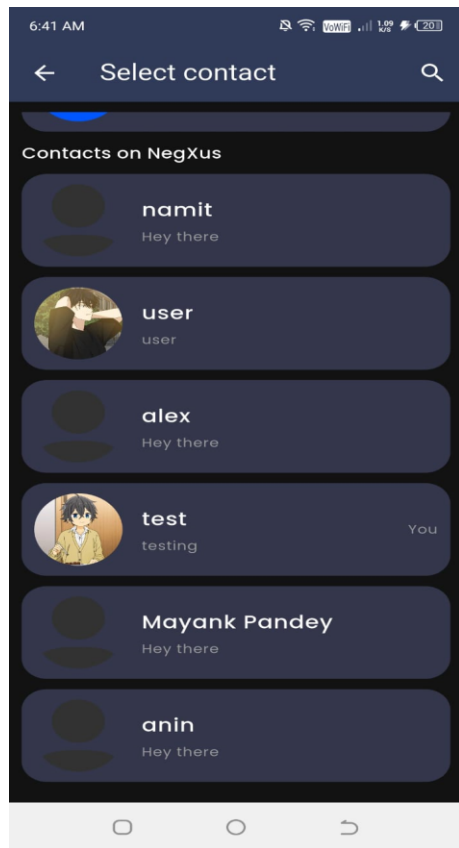
  @override
  Widget build(BuildContext context) {
    RxBool isEdit = false.obs;
    ProfileController profileController = Get.put(ProfileController());
    TextEditingController name = TextEditingController(text: profileController.currentUser.value.name);
    TextEditingController email = TextEditingController(text: profileController.currentUser.value.email);
    TextEditingController phone = TextEditingController(text: profileController.currentUser.value.phoneNumber);
    TextEditingController about = TextEditingController(text: profileController.currentUser.value.about);
    ImagePickerController imagePickerController = Get.put(ImagePickerController());
    RxString imagePath = "".obs;

    AuthController authController = Get.put(AuthController());

    return Scaffold(
      appBar: AppBar(
        title: Text("Profile"),
        actions: [
          IconButton(
            onPressed: () {
              authController.logoutUser();
            },
            icon: Icon(Icons.logout),
          ), // IconButton
        ],
      ), // AppBar
      body: Padding(
        padding: const EdgeInsets.all(10),

```

SNAPSHOTS:



LIMITATIONS

Despite the thoughtful design and robust implementation of the NegXus mobile chat application, several limitations exist due to development constraints, platform dependencies, and ongoing enhancements. These limitations help identify current gaps and guide future improvements.

1. Limited Scalability for High Traffic
 - While NegXus performs reliably for small to mid-sized user bases, the current Firebase-based architecture may require further optimization—such as Firestore indexing, function cold-start mitigation, and load balancing—for handling very large user volumes in real time.
2. Lack of Real-Time Voice/Video Calling
 - NegXus currently focuses on real-time text messaging. Voice or video calling features, though highly valuable for user engagement, are not yet implemented due to resource and time constraints.
3. Basic UI/UX Design
 - The application's current interface prioritizes functionality over polish. It lacks advanced animations, theme customization, and accessibility features (e.g., screen reader support, contrast settings), which may impact inclusivity and user appeal.
4. No End-to-End Encryption
 - Although Firebase provides secure data transmission, true end-to-end encryption (E2EE) for messages is not implemented. This limits the platform's privacy assurance for sensitive communications.
5. Limited File Sharing Features
 - Users can share basic media (e.g., images), but support for other file types (PDFs, docs, zip files) is restricted. File previews and in-app media playback are also minimal.
6. Absence of AI-Powered Features
 - NegXus currently does not include features like smart replies, chat summarization, sentiment analysis, or moderation bots, which could significantly enhance user experience and safety.
7. No Offline Messaging Support
 - Users cannot send messages while offline to be queued and delivered automatically when the connection is restored. This limits usability in areas with poor or intermittent internet access.
8. Lack of Third-Party Integrations
 - Integration with platforms like Google Drive, Dropbox, or calendar tools (for scheduling or file access) is currently missing, limiting ecosystem connectivity.
9. No Web or Desktop Support
 - NegXus is currently developed only for mobile (Android/iOS). Users cannot access chat via a web browser or desktop app, which may reduce flexibility and usage across devices.

ENHANCEMENTS

While **NegXus** currently delivers secure real-time messaging and media-sharing capabilities, there remains vast potential to enhance its performance, engagement, and feature set. The following proposed enhancements aim to address existing limitations and transform NegXus into a comprehensive, scalable, and user-friendly messaging platform.

1. **Cross-Platform Support (Web/Desktop)**

- Building web and desktop (Windows/macOS) versions of NegXus will allow users to seamlessly switch between devices and continue conversations across platforms, improving accessibility and convenience for professional and educational use.

2. **Voice and Video Calling**

- Integrating real-time voice and video calling using WebRTC or third-party APIs (like Agora or Twilio) will elevate the user experience by supporting richer, face-to-face interactions.

3. **End-to-End Encryption (E2EE)**

- Implementing E2EE for chat messages and media will significantly enhance user privacy and data security, making NegXus more competitive with industry leaders like Signal and WhatsApp.

4. **Offline Messaging and Caching**

- Supporting offline message composition and caching will ensure that messages are queued and sent once the user regains connectivity, enhancing reliability in low-network regions.

5. **AI-Powered Smart Features**

- Adding AI-driven capabilities such as smart replies, message summarization, language translation, and spam filtering can streamline communication and improve usability for diverse user groups.

6. **Advanced Media and File Sharing**

- Expanding support to include a broader range of file types (e.g., PDFs, docs, audio files) with in-app previews, cloud storage (via Firebase or Cloudinary), and size optimization will improve user workflows.

7. **Real-Time Notifications System**

- Enabling robust push notifications for new messages, mentions, or system updates will enhance user engagement. Customizable notification preferences can ensure non-intrusive communication.

8. **Custom Themes and UI Personalization**

- Providing users with options to customize themes, fonts, and color schemes can enhance visual appeal and accessibility, offering a more inclusive and personal chat experience.

9. **Admin and Moderation Tools**

- Enhancing backend tools for chatroom moderation, user role management, message deletion, and analytics dashboards will support educational and organizational use cases more effectively.

CONCLUSION

The development of **NegXus**, a real-time messaging platform, has been an exciting and insightful experience, blending mobile development, UI/UX design, and communication technologies. This project demonstrates how a responsive, feature-rich messaging application can be created using modern frameworks and technologies like **Flutter**, **Firebase**, and **Cloudinary** while prioritizing user engagement, security, and scalability.

Throughout the project, key aspects of mobile app development were explored and implemented, including **real-time chat functionality**, **user authentication**, **media sharing**, and **push notifications**. Each component contributed to the creation of a platform that not only facilitates seamless communication but also offers a visually appealing and responsive user experience.

The modular architecture of NegXus allows for future scalability and ease of maintenance. Future enhancements, such as **voice and video calls**, **end-to-end encryption**, and **mobile platform expansion**, are key areas to further improve user experience and overall app functionality. While features like **real-time collaboration tools** and **third-party integrations** were not fully implemented in the current version, the foundational work sets the stage for future upgrades.

A core strength of NegXus is its **user-centric design**, which ensures that the app remains intuitive, accessible, and focused on providing a smooth communication experience. This approach aligns with the growing demand for effective and secure mobile communication solutions, especially for professional and educational use.

In conclusion, **NegXus** serves as a comprehensive showcase of mobile app development principles applied to a real-time messaging context. It acts as both a valuable learning project and a stepping stone toward innovations in the realm of mobile communication, demonstrating the power of technology in fostering seamless, secure, and engaging interactions.

References

- **Flutter Documentation.** (n.d.). <https://flutter.dev/docs>
- **Firebase Documentation.** (n.d.). <https://firebase.google.com/docs>
- **Cloudinary Documentation.** (n.d.). <https://cloudinary.com/documentation>
- **Node.js Documentation.** (n.d.). <https://nodejs.org/en/docs/>
- **MDN Web Docs.** (n.d.). HTML, CSS, and JavaScript Guides. <https://developer.mozilla.org/>
- **FlutterFire Documentation.** (n.d.). <https://firebase.flutter.dev/docs/overview>
- **OpenAI Documentation.** (n.d.). <https://platform.openai.com/docs/>
- **Agora Documentation** (for video and voice calls). <https://www.agora.io/en/docs/>